Orma Project Report

Graph Hacks 2026 submission - 18 October 2026

Track 01 - Agents That Act on Connected Data

Project overview

Orma helps developers and learners find useful posts and videos saved across LinkedIn, Instagram, YouTube Watch Later and X. Instead of remembering a platform or exact wording, a user asks a question and follows the matching references back to their original sources.

The prototype uses Gemini to derive knowledge from available source content and FalkorDB as its primary persistent graph. The current local library contains 240 real indexed saves. Recording mode replays an import animation over this existing library and performs actual graph retrieval. It does not create invented resources or graph relationships.

Problem and user flow

Social platforms keep separate saved lists. A useful resource can become difficult to find when a person remembers its idea but forgets its source. Orma brings those references into one searchable workspace.

The user selects platforms, captures a capped batch through Linkosh, waits while available content is classified, explores the connected knowledge graph and asks a question. Each result provides a derived summary, original URL and an evidence path. A detail panel states the resource's analysis coverage.

Architecture and data handling

The frontend uses React, TypeScript and Vite. FastAPI coordinates ingestion, Gemini classification and graph retrieval. Linkosh works through the user's existing Chrome sessions, and account cookies remain in Chrome.

FalkorDBLite runs genuine FalkorDB locally in WSL. An optional Docker configuration supports a separate FalkorDB server. The graph stores source URLs, platform and author metadata, derived summaries, concepts and relationships. Raw post bodies and media are processed temporarily rather than retained as new source archives. Existing historical captures are separate from the Orma knowledge store.

Graph data model

User SAVED Resource; Resource FROM_PLATFORM Platform; Author CREATED Resource.

Resource ABOUT Concept represents a topic.

Resource MENTIONS Concept represents an entity.

Resource HELPS_WITH Concept represents a goal.

Concept RELATED_TO Concept includes a reason and supporting resource ID.

Search RETRIEVED Resource records search memory across sessions.

Gemini receives the existing graph vocabulary to encourage consistent concept names across platforms. Concepts come from incoming content rather than a fixed taxonomy.

Retrieval and core Cypher

Direct matching identifies resources attached to relevant concepts:

```
MATCH (r:Resource)-[e:ABOUT|MENTIONS|HELPS_WITH]->(c:Concept)
WHERE any(term IN $terms
  WHERE toLower(c.name) CONTAINS term
  OR term CONTAINS toLower(c.name))
AND ($platform = '' OR r.platform = $platform)
RETURN r.id, c.name, type(e)
```

Two-hop expansion adds resources connected through a related concept:

```
MATCH (a:Concept)-[:RELATED_TO]-(b:Concept)<-[:ABOUT]-(r:Resource)
WHERE any(term IN $terms
  WHERE toLower(a.name) CONTAINS term
  OR term CONTAINS toLower(a.name))
AND ($platform = '' OR r.platform = $platform)
RETURN DISTINCT r.id, a.name, b.name
```

The backend combines graph evidence with a lexical match over derived titles and summaries. It ranks results and surfaces strong matches from different platforms. Queries use parameters, and the model cannot execute arbitrary generated Cypher. Normal search uses Gemini for intent interpretation and grounded synthesis. Recording search uses deterministic terms and retrieved summaries to avoid live model delays.

Current library and validation

LinkedIn: 100 saved posts.

Instagram: 40 saved items returned by the available capture.

YouTube Watch Later: 50 videos.

X: 50 bookmarks.

Total: 240 real indexed references.

A browser check asked for saved resources about Software Engineering Career across all platforms. It returned 12 resources with source links and real graph evidence paths covering all four platforms. The recording search completed in 102 ms in one local verification run; this is an observation rather than a performance guarantee.

The production frontend build passed. Three backend tests passed, covering ingestion caps, Watch Later filtering, canonical URLs, real FalkorDB traversal, parameterized query safety, updates and avoiding retained raw text. Earlier checks passed 11 Linkosh memory-ingestion tests and two bridge tests. Browser verification passed platform selection, the replay progress flow, graph rendering, library filtering, clickable citations, source details and mobile layout without overflow or JavaScript errors.

Recording scope and remaining work

The provided 2 minute 43 second video combines slides with the app walkthrough and a generic synthetic voiceover. The import loading sequence is explicitly labeled as a replay of the existing library. Search and

graph connections use actual stored data.

Requested capture caps are LinkedIn 100 and 50 each for Instagram, YouTube and X. Instagram returned 40 items; the prototype does not invent the remaining ten. Direct YouTube analysis and image-preview analysis depend on access and model quotas. Some resources use metadata only, and coverage appears in the interface. Background video enrichment is disabled for recording.

This is a local, single-user prototype. Session-based platform integrations may change. Future work includes robust incremental capture, broader media handling, stronger concept consolidation, account isolation and deployment. A new checkout starts without the user's private graph and must ingest authorized content.

Reproduction and source

The public repository is https://github.com/ShibuJoseph18/orma. README.md contains setup instructions, the data model, validation commands and instructions for rebuilding the Linkosh bridge. backend/graph.py contains the product's Cypher queries; backend/test_core.py exercises graph retrieval and data retention.

Run in Linux or WSL with Node.js 20.19 or later and Python 3.12 or later. Install backend requirements, configure GEMINI_API_KEY in the ignored backend/.env file, install frontend dependencies and run bash scripts/start.sh. Use /?live=1 for live ingestion. DEMO.md describes the prepared recording flow.